

ETL vs ELT: Data Pipeline Transformation Patterns

 systemdesignschool.io/fundamentals/etl-vs-elt

ETL: Extract → Transform → Load



Transform outside warehouse. Raw data not preserved.

ETL vs ELT

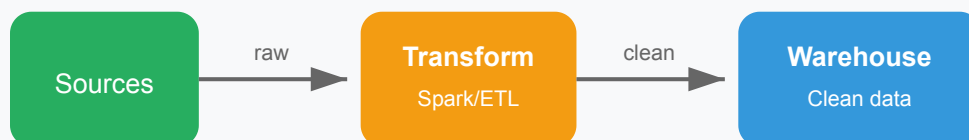
Data pipelines move data from source systems to analytical destinations. A fundamental question shapes pipeline architecture: where do you transform the data?

The answer has shifted dramatically as storage costs dropped and cloud warehouse compute became elastic. Understanding both patterns helps you choose the right approach for your constraints.

Traditional ETL: Transform Before Loading

ETL (Extract-Transform-Load) processes data in a separate transformation layer before loading into the destination.

ETL: Extract → Transform → Load



Transform outside warehouse. Raw data not preserved.



1. **Extract** raw data from sources—databases, APIs, log files, third-party services
2. **Transform** in a processing system (Spark, custom scripts, ETL tools)—clean malformed records, deduplicate, normalize schemas, aggregate, join across sources
3. **Load** the refined, ready-to-use result into the data warehouse

This pattern emerged from historical constraints. Data warehouse storage was expensive—you paid per terabyte stored. Warehouse compute was limited—running complex transformations in the warehouse was slow and costly. Transforming before loading reduced both storage and

query costs.

Strengths:

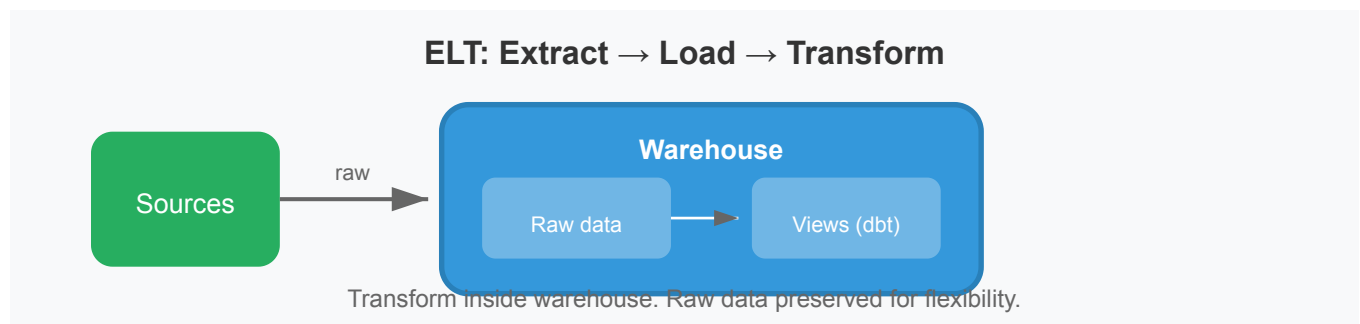
- Warehouse stays clean with pre-processed, validated data
- Query performance is better because data arrives optimized for common access patterns
- Storage costs stay lower—you don't store raw data that may never be queried
- Sensitive fields can be filtered or masked before entering the warehouse

Limitations:

- Raw data is transformed away—if you realize you need a field you didn't include, you must re-extract from the source
- Schema changes (new columns, format changes) require pipeline modifications before data can flow
- Transformation logic lives outside the warehouse, often in tools with different development workflows than SQL

Modern ELT: Load First, Transform In Place

ELT (Extract-Load-Transform) inverts the order. Load raw data into the warehouse, then transform using the warehouse's own compute.



1. **Extract** raw data from sources
2. **Load** directly into the warehouse, preserving original format (often as JSON or in staging tables)
3. **Transform** using warehouse-native tools—SQL queries, dbt models, stored procedures

The economics flipped. Cloud storage became cheap (pennies per GB/month). Cloud warehouses like BigQuery and Snowflake provide elastic compute that scales instantly. The constraints that drove ETL largely disappeared.

Strengths:

- Full flexibility—raw data is preserved, so new requirements don't need re-extraction
- Schema evolution is easier—new fields automatically appear in raw tables
- Multiple teams can derive different views from the same raw data

- Transformations are SQL (which analysts already know) and version-controlled alongside documentation

Limitations:

- Storage grows faster—you keep raw data plus transformed views
- Raw data may contain sensitive fields that shouldn't persist (PII, credentials)
- Query costs can increase if users accidentally scan large raw tables

Concrete Comparison: Adding a New Metric

Suppose marketing wants a new metric: "average order value by customer segment." The source data includes orders, customers, and segment assignments.

ETL approach:

1. Modify the extraction job to include segment data (if not already extracted)
2. Update transformation scripts to calculate the new aggregation
3. Deploy, wait for the next pipeline run, validate results
4. If something's wrong with the calculation, re-run the entire pipeline

ELT approach:

1. Raw order, customer, and segment data is already in the warehouse
2. Write a SQL transformation (or dbt model) that joins and aggregates
3. Run it—results available in minutes
4. If the calculation needs tweaking, just modify the SQL and re-run

The ELT approach is faster to iterate because the data is already available. The transformation is just a SQL query that anyone with warehouse access can write and test.

When to Use Each

Scenario	ETL	ELT
Strict compliance—raw data can't persist	ETL	
Sensitive data requiring masking before storage	ETL	
Very cost-sensitive with large data volumes	ETL	
Evolving requirements with exploratory analysis		ELT
Data science teams needing raw access		ELT
Modern cloud warehouse (Snowflake, BigQuery, Databricks)		ELT

Many organizations use hybrid approaches: ETL for sensitive data that requires transformation before storage, ELT for everything else.

The Modern Data Stack

The "modern data stack" typically follows an ELT pattern:

Extract & Load: Tools like Fivetran, Airbyte, or Stitch connect to sources (SaaS APIs, databases, event streams) and load raw data into the warehouse with minimal transformation.

Transform: dbt (data build tool) runs SQL-based transformations inside the warehouse. Models are version-controlled, tested, and documented alongside business logic.

Warehouse: Snowflake, BigQuery, Databricks, or Redshift store both raw and transformed data with elastic compute.

Raw data persists as the "source of truth." Transformations are just SQL that can be re-run anytime. Adding a new metric or fixing a bug means modifying a dbt model, not rebuilding the entire pipeline from source extraction.

What's Next

Loading data is just the first step. When source data changes and downstream systems need updates, **backfill and reprocessing** handle historical corrections and schema evolution.